CS50's Introduction to Cybersecurity

Lecture 1 — Securing Data

Harvard University | David J. Malan | cs50.harvard.edu/cybersecurity

Contents

1. Securing Data Overview	8. Cryptanalysis
2. Passwords & Storage Risks	9. Public-Key Cryptography
3. Hashing	10. Key Exchange & Diffie-Hellman
4. Salting	11. Digital Signatures
5. One-Way Hash Functions	12. Passkeys / WebAuthn
6. Codes & Encoding	13. Encryption in Transit
7. Ciphers, Keys & Secret-Key Cryptography	14. Deletion, Full-Disk Encryption & Quantum

1. Securing Data — Overview

This lecture builds on Week 0's focus on account security and shifts the lens to how data itself is stored and transmitted securely. A critical insight is that data security does not rest solely with end-users — third parties such as web services and cloud providers are always involved in storing sensitive information.

The central problem: when a system stores usernames and passwords in plain text, any adversary who gains access to that file immediately possesses all credentials. This lecture introduces the layered technical tools designed to mitigate that risk.

> **Key Insight:** *Security is fundamentally a cost-raising exercise against adversaries. Every layer added — hashing, salting, encryption — increases the time, computational resources, and expertise required to breach a system.*

2. Passwords & Storage Risks

Passwords are the most common mechanism for authenticating users. However, the naive approach of storing them in plain text is catastrophically insecure. If an adversary obtains the storage file — through a data breach, SQL injection, or insider threat — every user account is immediately compromised.

This motivates the need for a transformation applied to passwords before storage, so that the stored value reveals nothing about the original password.

> **Key Insight:** *The question is never 'will a system be breached?' but 'what can an adversary do with what they find?' Proper password storage limits the damage of any inevitable breach.*

3. Hashing

What Is Hashing?

Hashing is the process of converting plaintext input (such as a password) into a fixed-length, opaque output called a hash value, using a deterministic mathematical function called a hash function. The transformation is designed to be:

- Fast to compute in the forward direction (plaintext → hash).
- Computationally infeasible to reverse (hash → plaintext).
- Unpatterned — the output should appear random and reveal nothing about the input.

How Login Verification Works

Because hashing is deterministic, the same input always produces the same hash. Systems exploit this for password verification:

- At registration: the user's password is hashed, and only the hash is stored.
- At login: the entered password is hashed again and compared to the stored hash. No plaintext is ever retained.

Attacks Against Hashing

Hashing alone does not guarantee security. Several attack vectors exist:

- **A precomputed list of common words and passwords is hashed using the target function, and results are compared against the database.:** Dictionary Attack
- **Every possible character combination is systematically hashed and compared. Feasibility depends on password length, character set, and hash speed.:** Brute-Force Attack
- **Precomputed lookup tables mapping hash outputs back to inputs for a specific algorithm. Can cover billions of password-hash pairs, though they require terabytes to petabytes of storage.:** Rainbow Tables

> **Key Insight:** *The existence of rainbow table attacks highlights a fundamental weakness: if two users share a password, they share a hash. An attacker who cracks one hash instantly cracks all identical ones.*

4. Salting

Salting is the defense against rainbow table and dictionary attacks. A salt is a unique, randomly generated value that is appended or prepended to a user's password before hashing.

The resulting hash is: Hash(password + salt). Both the salt and the resulting hash are stored. Crucially, even two users with identical passwords will produce entirely different hashes because their salts differ.

- Salting invalidates precomputed rainbow tables, because the adversary would need a separate table per possible salt value — an impractical computational requirement.
- It forces attackers to attack each password hash individually rather than exploiting shared hashes.
- NIST (the National Institute of Standards and Technology) explicitly recommends that memorized secrets — i.e., passwords — be both hashed and salted.

> **Key Insight:** *Salting does not make hashing faster or the hash longer. Its sole purpose is to ensure uniqueness per user, dramatically increasing the adversarial workload.*

5. One-Way Hash Functions

One-way hash functions are a specialized class of hash functions with additional security properties beyond basic hashing. They accept a string of arbitrary length as input and always produce a hash output of a fixed, predetermined length.

The defining property is irreversibility: even the system that stores the hash and knows the hash function cannot reconstruct the original password from the hash value alone.

An important mathematical phenomenon: hash collisions. Because the input space is infinite (any string of any length) and the output space is finite (a fixed-length hash), different inputs can — in theory — produce identical hash outputs. Robust cryptographic hash functions minimize collision probability to the point of practical impossibility.

> **Key Insight:** *Cryptographic hash functions used in modern systems (such as bcrypt, scrypt, Argon2, and SHA-256) are specifically engineered to be computationally expensive, reducing brute-force feasibility even on modern hardware.*

6. Codes & Encoding

Cryptography is the academic discipline concerned with techniques for transmitting information securely between parties in the presence of adversaries. Codes are one of its foundational tools.

- **The transformation of plaintext into codetext using a substitution or representation scheme.:** Encoding
- **The reverse process — converting codetext back into plaintext.:** Decoding

Codes are often fixed and rely on secrecy of the code book rather than an algorithmic key. They are distinct from ciphers, which use mathematical keys.

7. Ciphers, Keys & Secret-Key Cryptography

Ciphers

Ciphers operate at an algorithmic level. A cipher transforms plaintext into ciphertext through a process called encryption. The reverse process — recovering plaintext from ciphertext — is called decryption. Unlike codes, ciphers use parameterized algorithms: the same algorithm can produce different outputs based on the key provided.

Keys

A cryptographic key is a large numeric value used as a parameter in encryption and decryption algorithms. The security of modern cryptographic systems rests not on keeping the algorithm secret (most algorithms are published), but on keeping the key secret.

Secret-Key Cryptography (Symmetric Encryption)

In secret-key (symmetric) cryptography, the same key is used for both encryption and decryption. Both the sender and receiver must possess this shared secret. The model is:

- Sender: Encrypt(plaintext, key) → ciphertext
- Receiver: Decrypt(ciphertext, key) → plaintext

The critical weakness is the key distribution problem: how do two parties securely share the key if they have never communicated before and there is no trusted channel?

> **Key Insight:** *Symmetric encryption is computationally fast and efficient for large data, which is why it is used for bulk data encryption. The challenge is always key distribution.*

8. Cryptanalysis

Cryptanalysis is the complementary field to cryptography — where cryptography builds secure systems, cryptanalysis studies how to break them. Practitioners analyze encryption algorithms, protocols, and implementations to identify weaknesses.

Cryptanalysis informs the development of stronger algorithms. Historical examples include breaking the Enigma cipher during World War II and the deprecation of DES (Data Encryption Standard) after cryptanalysts demonstrated its vulnerability to brute-force attacks.

> **Key Insight:** *Understanding how attacks work is prerequisite to building defenses. This is the dual-use nature of cryptanalysis — the same knowledge that enables offense enables stronger defense.*

9. Public-Key Cryptography (Asymmetric Encryption)

Asymmetric encryption resolves the key distribution problem of symmetric cryptography. Instead of one shared key, each party holds a mathematically related key pair:

- **Freely distributed to anyone. Used to encrypt data intended for the key pair's owner.:** Public Key
- **Kept secret by the owner. Used to decrypt data encrypted with the corresponding public key.:** Private Key

The mathematical relationship ensures that data encrypted with a public key can only be decrypted with the paired private key, and vice versa. Knowing the public key does not allow an adversary to derive the private key.

RSA

RSA (Rivest–Shamir–Adleman) is the most widely adopted asymmetric encryption standard. Its security is grounded in the computational hardness of factoring very large integers — a problem for which no efficient classical algorithm is known. RSA underpins HTTPS, digital certificates, and many other widely deployed security protocols.

> **Key Insight:** *Asymmetric encryption made secure internet commerce possible. Without it, two strangers cannot safely exchange a symmetric key over an untrusted network — making online banking and e-commerce impossible.*

10. Key Exchange — Diffie-Hellman

The Diffie-Hellman protocol solves the key exchange problem: how can two parties establish a shared secret over a public channel without ever transmitting the secret itself?

The protocol uses modular arithmetic with two publicly agreed-upon parameters: a base value g and a large prime p. Each party independently generates a private key and uses it with g and p to compute a public value. By exchanging public values, both parties can independently compute the same shared secret s — yet an eavesdropper observing all transmitted values cannot derive s without solving the discrete logarithm problem, which is computationally infeasible for sufficiently large parameters.

- The shared secret s is never transmitted directly.
- Security relies on the mathematical difficulty of the discrete logarithm problem.
- Diffie-Hellman is used in TLS (the protocol securing HTTPS) for forward secrecy.

> **Key Insight:** *Forward secrecy is an important property: even if a server's private key is later compromised, past sessions encrypted via Diffie-Hellman key exchange cannot be decrypted retroactively.*

11. Digital Signatures

Digital signatures allow the recipient of a document or message to verify both its authenticity (it came from who it claims to be from) and its integrity (it was not altered in transit). The process uses both hash functions and asymmetric key pairs.

Signing Process

- **The message content is passed through a cryptographic hash function, producing a hash value.:** Step 1
- **The sender's private key and the hash value are passed to a digital signature algorithm, producing a digital signature that is attached to the message.:** Step 2

Verification Process

- **The recipient independently hashes the received message content.:** Step 1
- **The sender's public key and the received signature are fed into the decryption algorithm, producing the sender's original hash value.:** Step 2
- **If both hash values match, the signature is valid: the message is authentic and unmodified.:** Step 3

> **Key Insight:** *Digital signatures provide non-repudiation — a sender cannot later deny signing a document, because only their private key could have produced the valid signature. This is legally significant in many jurisdictions.*

12. Passkeys & WebAuthn

Passkeys (standardized under the WebAuthn protocol) represent the next evolution in authentication, moving beyond the inherent weaknesses of passwords. They are increasingly supported by major operating systems and web platforms.

How Passkeys Work

Passkeys are device-bound cryptographic key pairs generated at account creation:

- The device generates a public key and a private key.
- The public key is sent to and stored by the website or service.
- The private key never leaves the device.

Authentication Flow

When logging in, the service issues a challenge value. The device signs this challenge with the stored private key, producing a signature. The service verifies the signature using the stored public key. If valid, authentication succeeds.

Because no shared secret (password) is transmitted or stored on the server, phishing attacks and credential database breaches become significantly less effective.

> **Key Insight:** *Passkeys are resistant to phishing because the private key never leaves the device and the credential is bound to the specific website's origin — a fake site cannot request or receive a valid passkey signature.*

13. Encryption in Transit

Encryption in transit protects data as it moves across networks — from a user's device to a server and back. Without it, any network participant along the route (ISPs, routers, malicious actors on the same Wi-Fi) can read the transmitted data.

The Man-in-the-Middle Threat

A man-in-the-middle (MITM) attack occurs when a third party intercepts communications between two parties, potentially reading or altering the data without either party's knowledge. Encryption in transit makes intercepted data unreadable.

Third-Party Intermediaries

Even with encryption in transit, intermediary services — such as email providers — may have access to the content of communications. The provider encrypts the connection between the user and its own servers, but the provider itself can read the stored data.

End-to-End Encryption (E2EE)

End-to-end encryption ensures that only the communicating parties — not the service provider or any intermediary — can read the content. Messages are encrypted on the sender's device and can only be decrypted by the recipient's device. This is employed by messaging applications such as Signal and by certain encrypted email protocols.

> **Key Insight:** *The distinction between 'encrypted in transit' and 'end-to-end encrypted' is critical for privacy. Many services advertise encryption while still being able to read user content. True E2EE removes that capability from providers.*

14. Deletion, Full-Disk Encryption & Quantum Computing

File Deletion

Deleting a file on a conventional operating system does not erase the underlying data. The OS typically only removes the file's directory entry — it 'forgets' where the file lives — but the actual data remains on the storage medium until overwritten by new files.

This means forensic tools can recover deleted files from 'free' space on a drive, posing significant privacy and security risks.

Secure Deletion

Secure deletion overwrites the storage locations occupied by deleted data with zeros, ones, or random sequences of bits, preventing forensic recovery. Secure deletion tools implement this overwriting before freeing the space.

Full-Disk Encryption (Encryption at Rest)

Full-disk encryption (FDE) encrypts every bit of data stored on a device's storage medium. The data is decrypted on-the-fly only when the correct credentials are provided at boot.

- **If the device is stolen or sold, the encrypted data is unreadable without the decryption key.:** Benefits
- **If the user forgets the password or authentication fails (e.g., facial recognition after significant facial change), access to data is permanently lost.:** Risk — Lost Credentials
- **Adversaries exploit the same encryption technology to lock victims out of their own devices. Ransomware encrypts the victim's data and demands payment for the decryption key.:** Risk — Ransomware

Quantum Computing

Quantum computing leverages quantum mechanical phenomena (superposition, entanglement) to perform certain computations exponentially faster than classical computers. Its implications for cybersecurity are profound:

- **Shor's algorithm, running on a sufficiently powerful quantum computer, could factor large integers efficiently — breaking RSA encryption.:** Threat to RSA
- **Grover's algorithm provides a quadratic speedup for brute-force searches, effectively halving the security of symmetric keys.:** Threat to Password Security
- **The critical question is whether adversaries (nation-states, well-funded organizations) will gain access to cryptographically relevant quantum computers before defenders can transition to quantum-resistant cryptographic standards.:** The Timeline Concern

> **Key Insight:** *NIST has already standardized a suite of post-quantum cryptographic algorithms designed to remain secure against quantum attacks. The transition to these standards is one of the most significant cryptographic challenges of the coming decade.*

Summary — Key Concepts

Concept	What It Does	Key Limitation / Threat
Hashing	Converts passwords to irreversible fixed-length values	Dictionary & brute-force attacks; rainbow tables
Salting	Adds random value before hashing for uniqueness	Increases attacker workload, does not stop all attacks
One-Way Hash Functions	Cryptographically irreversible hash; fixed output	Hash collisions (minimized in modern functions)
Secret-Key Cryptography	Encrypts data with a shared key	Key distribution problem
Public-Key Cryptography	Encrypts with public key; decrypts with private key	Slow for large data; quantum threatens RSA
Diffie-Hellman	Establishes shared secret without transmitting it	Vulnerable to quantum (discrete log problem)

Digital Signatures	Authenticates origin and integrity of documents	Private key compromise invalidates all signatures
Passkeys / WebAuthn	Device-bound key pair; phishing-resistant auth	Device loss requires recovery mechanism
End-to-End Encryption	Only communicating parties can read data	Provider cannot assist in law enforcement decryption
Secure Deletion	Overwrites deleted data with random bits	Not default behavior; requires explicit tools
Full-Disk Encryption	Encrypts entire storage device	Lost key = permanent data loss; enables ransomware
Quantum Computing	Exponentially faster computation for some problems	Threatens RSA, hash security; solution: post-quantum crypto